Passing Arguments to Methods

Lecture 8

Based on Slides of Dr. Norazah Yusof

Method

- A method is a collection of statements that are grouped together to perform an operation.
- A method is always defined inside a class.
- *Method signature* or *method header* is the combination of the method name and the parameter list.

Method definitions have the form

```
method header {
    method body
}
```

Parts of a Method Header

Method Modifiers | Return Type | Method Name | Parentheses

```
public int getEmpNum ()
 {

     return empnum;

 }
```

The above method is an instance method.
The type of result returned is *returnType* where int means the method returns an integer result.

Parts of a Method Header

Method Modifiers | Return Type | Method Name | Parentheses

```
 public void setEmpNum (int num)
  {

      empnum = num;

  }
```

The above method is an instance method.
The type of result returned is *returnType* where void means the method does not return a result.

return Statement

- A method, unless void, returns a value of the specified type to the calling method.
- The return statement is used to immediately quit the method and return a value:

return *expression*;

The type of the return value or expression must match the method's declared return type.

return Statement (cont'd)

- A method can have several return statements; then all but one of them must be inside an if or else (or in a switch):

```
public someType myMethod (...)
{
    ...
    if (...)
        return <expression1>;
    else if (...)
        return <expression2>;
    ...
    return <expression3>;
}
```

return Statement (cont'd)

- A boolean method can return true, false, or the result of a boolean expression:

```
public boolean myMethod (...)
{
   ...
   if (...)
      return true;
   ...
   return n % 2 == 0;
}
```

Calling an instance method

- A method executes when it is called.
- The main method is automatically called when a program starts, but other methods are executed by method call statements
- A method with a non-void return value type can also be invoked as a statement.
- An invocation of an instance method that returns an integer value may use an assignment statement or output statement:

```
int vnum;
vnum = obj1.getEmpNum();
```

or

```
System.out.println ("employee num:" +
   obj1.getEmpNum());
```

Calling an instance method

- An invocation of a `void` instance method is simply a statement:

```
objectName.methodName();
```

- Example:

```
obj1.setEmpNum(123);
```

Passing Parameters to Constructors

- Any expression that has an appropriate data type can serve as a parameter:

```
public static void main (String[] args) {
    Employee emp1 = new Employee();
    Employee emp2 = new Employee(123); // argument
    Employee emp3= new Employee(456, "Ahmad"); // argument
}
```

```
public class Employee {
   public Employee () {  }
   public Employee (int num) {  // parameters
       empnum = num ;}
   public Employee (int num, String name) {
       empnum = num ;
       empname = name;
    }
```

Arguments are Passed by Value

- In Java, all arguments of the primitive data types are *passed by value*, which means that only a copy of an argument's value is passed into a parameter variable.
- Any change to a parameter variable inside a method, has no affect to the original argument.

Passing Parameters : primitive type

- Primitive data types are always passed "<u>by value</u>": the value is copied into the parameter.

```
int x = 123;
emp1.setEmpNum ( x );
```

```
public class Employee
{
   ...
   public void setEmpNum (int num)
   {
      empnum = num;
      ...
   }
}
```

copy

num acts like a local variable in **setEmpNum**

x: **123** — copy → num: **123**

Passing Parameters (cont'd)

```
public class Test
{
   public double square (double x)
   {
      x *= x;
      return x;
   }

   public static void main(String[ ] args)
   {
      Test calc = new Test ();
      double x = 3.0;
      double y = calc.square (x);
      System.out.println (x + "  " + y);
   }
}
```

x here is a copy of the parameter passed to **square**. The copy is changed, but...

... the original **x** is unchanged. Output: **3 9**

Passing Objects as Arguments

- Objects can be passed to methods as arguments.
- Java passes all arguments *by value*.
- When an object is passed as an argument, the value of the reference variable is passed.
- The value of the reference variable is an address or reference to the object in memory.
- When a method receives a reference variable as an argument, it is possible for the method to modify the contents of the object referenced by the variable.

Passing Object As Arguments to Methods

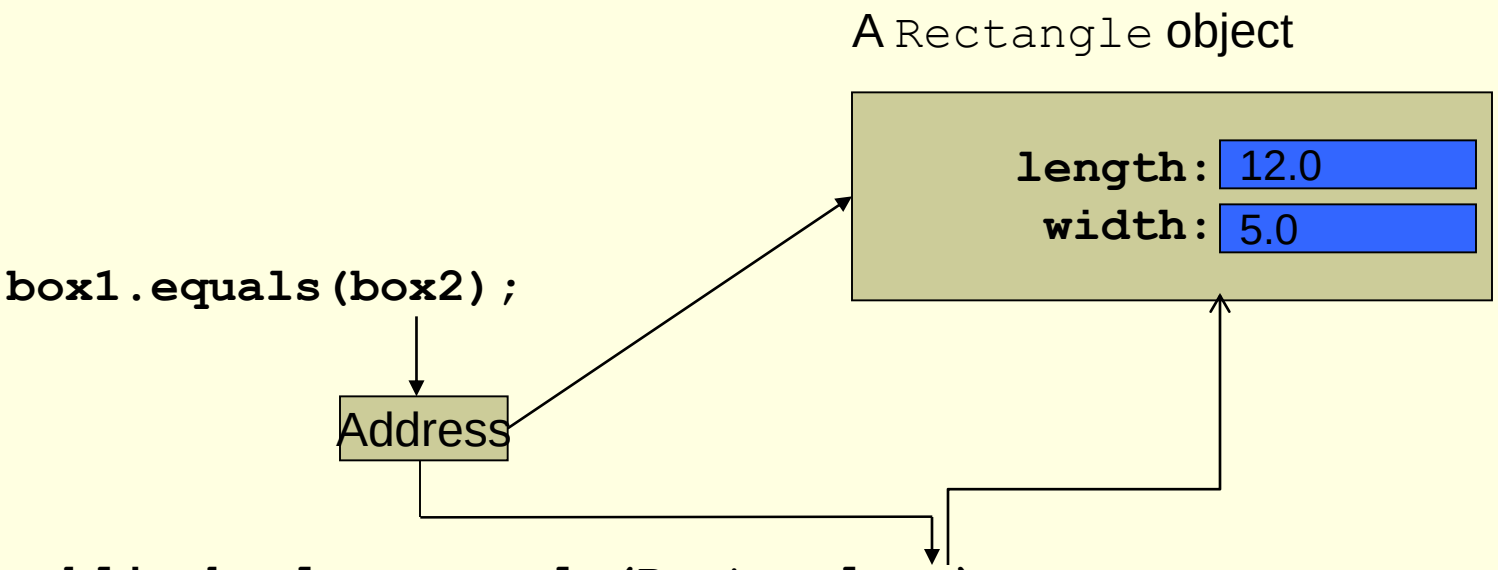
```
public class Rectangle {
  private double width;
  private double length;

  public Rectangle (double w,
                    double l) {
    width = w;
    length = l;
  }

  public void setWidth(double w) {
    width = w;
  }

  public void setLength(double l) {
    length = l;
  }

  public double getWidth() {
    return width;
  }

  public double getLength() {
    return length;
  }

  public double calcArea() {
    return width * length;
  }

  public boolean equals(Rectangle r) {
    return (this.calcArea() == r.calcArea());
  }
}
```

Passing Object As Arguments to Methods

```
1.  public class PassObject  {
2.     public static void main(String[ ] args)  {
3.          Rectangle box1 = new Rectangle(12.0, 5.0);
4.          Rectangle box2 = new Rectangle(10.0, 8.0);
5.          if (box1.equals(box2))
6.             System.out.println ("Yes, it's equals");
7.          else
8.             System.out.println ("No, it's not equal");
9.       }
10.  }
```

Line 3 and 4 creates two instances of Rectangle class.
Line 5 the value of the variable box2 (reference variable of Rectangle) is passed as an argument to the instance method, equals().
In Rectangle class, equals() method has a parameter, r, which is a Rectangle reference variable and return a boolean value, after comparing whether the two objects are equal.

Passing Objects as Arguments




```
public boolean equals(Rectangle r)
{
   return (this.calcArea() == r.calcArea());
}
```

Returning Objects From Methods

```
import java.util.*;

public class PassObject3  {

   public static void main(String[ ] args)  {
      Rectangle box3 = new
                        Rectangle().inputRectangle();
      System.out.println (“The area: “ +
                        box3.calcArea());

   }
```

```
public Rectangle inputRectangle() {
  Scanner inp = new Scanner (System.in);
  double newWidth, newLength;

  System.out.print ("Enter width : ");
  newWidth = inp.nextDouble();
  System.out.print ("Enter length : ");
  newLength = inp.nextDouble();

  return new Rectangle(newWidth, newLength);
  }
}
```

Returning Objects From Methods

- Methods can return references to objects.
- Just as with passing arguments, a copy of the object is not returned, only its address.
- Method return type:

```
public Rectangle inputRectangle ()
{
      …
     return new Rectangle(newWidth, newLength);
}
```

The `toString` Method

- All objects have a `toString` method that returns the class name and a hash of the memory address of the object.
- We can override the default method with our own to print out more useful information.

The `equals` Method

- When the `==` operator is used with reference variables, the memory address of the objects are compared.
- The contents of the objects are not compared.
- All objects have an `equals` method.
- The default operation of the `equals` method is to compare memory addresses of the objects (just like the `==` operator).

The `equals` Method

- The `Rectangle` class has an `equals` method.
  - Write a method that only the addresses of the objects are compared
- Now, instead of using the `==` operator to compare two `Rectangle` objects, we should use the `equals` method.
  - Compare objects by their contents rather than by their memory addresses

Methods That Copy Objects

- There are two ways to copy an object.
  - You cannot use the assignment operator to copy reference types
  - Reference only copy
    - This is simply copying the address of an object into another reference variable.
  - Deep copy (correct)
    - This involves creating a new instance of the class and copying the values from one object into the new object

Copy Constructors

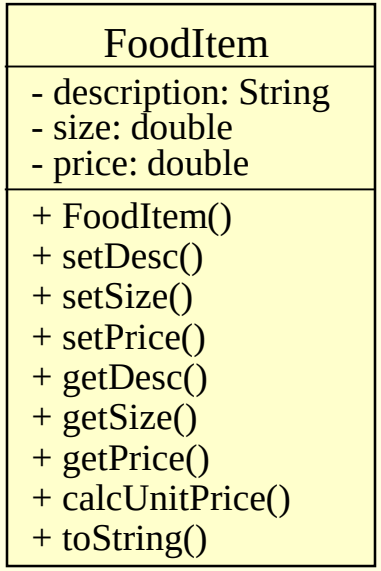
- A copy constructor accepts an existing object of the same class and clones it

```
public Rectangle(Rectangle object2)
{
    this.width = object2.width;
    this.length = object2.length;
}

// Create a rectangle object, rect1
Rectangle rect1 = new Rectangle(5.0, 4.5);
//Create rect2, a copy of rect1
 Rectangle rect2 = new Rectangle(rect1);
```

The FoodItem class

The class diagram that represents a food item at a market

UML class diagram



Methods for Class FoodItem

- For most classes, we need to provide methods that perform tasks similar to the ones below.
- a method to create a new FoodItem object with a specified description, size, and price (*constructor*)
- methods to change the description, size, or price of an existing FoodItem object (*mutators*)
- methods that return the description, size, or price of an existing FoodItem object (*accessors*)
- a method that returns the object state as a string (a **`toString()`** method).

Method Headers for Class FoodItem

Method	*Description*
`public FoodItem(String, double, double)`	Constructor - creates a new object whose three data fields have the values specified by its three arguments.
`public void setDesc(String)`	Mutator - sets the value of data field **description** to the value indicated by its argument. Returns no value.
`public void setSize(double)`	Mutator - sets the value of data field **size** to the value indicated by its argument. Returns no value.

Method Headers for Class FoodItem, cont'd.

Method	*Description*
`public void setPrice(double)`	Mutator - sets the value of data field **price** to the value indicated by its argument. Returns no value.
`public String getDesc()`	Accessor - returns a reference to the string referenced by data field **description**.
`public double getSize()`	Accessor - returns the value of data field **size**.

Method Headers for Class FoodItem, cont'd.

Method	*Description*
`public double getPrice()`	Accessor - returns the value of data field **`price`**.
`public String toString()`	Returns a string representing the state of this object.
`public double calcUnitPrice()`	Returns the unit price (price / size) of this item.

Class FoodItem

```
/*
 * FoodItem.java
 * Represents a food item at a market
 */
public class FoodItem {

  // data fields
  private String description;
  private double size;
  private double price;
```

Class FoodItem, cont'd.

```
// methods
// postcondition: creates a new object with data
//   field values as specified by the arguments
public FoodItem(String desc, double aSize,
                  double aPrice) {
   description = desc;
   size = aSize;
   price = aPrice;
}

// postcondition: sets description to argument
//   value
public void setDesc(String desc) {
   description = desc;
}
```

Class FoodItem, cont'd.

```
// postcondition: sets size to argument value
public void setSize(double aSize) {
   size = aSize;
}

// postcondition: sets price to argument value
public void setPrice(double aPrice) {
   price = aPrice;
}

// postcondition: returns the item description
public String getDesc() {
   return description;
}

// postcondition: returns the item size
public double getSize() {
   return size;
}
```

Class FoodItem, cont'd.

```
  // postcondition: returns the item price
  public double getPrice() {
     return price;
  }

  // postcondition: returns string representing
  //  the item state
  public String toString() {
     return description + ", size : " + size +
            ", price $" + price;
  }

  // postcondition: calculates & returns unit price
  public double calcUnitPrice() {
     return price / size;
  }
}
```

void Methods

- The mutator methods are **`void`** methods.

```
public void setSize(double aSize) {
    size = aSize;
}
```

- Method **`setSize()`** sets the value of data field **`size`** to the value passed to parameter **`aSize`**.
- A **`void`** *method is executed for its effect* because it does not return a value; however, it may change the state of an object or cause some information to be displayed.

The return Statement

- Each **`FoodItem`** method that is not a **`void`** method or a constructor returns a single result.

```
public double getPrice() {
    return price;
}
```

The **return statement** above returns the value of data field **`price`** as the function result.

- Form: **`return`** expression;
- Example: **`return x + y;`**
- Interpretation: The expression after the keyword **`return`** is returned as the method result. Its data type must be assignment compatible with the type of the method result.

toString() Method

- A **`toString()`** method returns a reference to a **`String`** object that represents an object's state.

```
public String toString() {
   return description + ", size : " + size +
          ", price $" + price;
}
```

The expression after **`return`** specifies the characters stored in the **`String`** object returned as the method result. It starts with the characters in **`description`** followed by the characters **`, size :`** followed by characters representing the value of data field **`size`** and **`price`**; e.g.,

`"apples, Macintosh, size: 2.5, price $1.25"`

Calling methods

- Because a **`void`** method does not return a value, a call to a void method is a Java statement all by itself. If **`myCandy`** is type **`FoodItem`**, an example would be

```
myCandy.setDescription("Snickers, large");
```

- Because a non-**`void`** method returns a value, a call to a non-**`void`** method must occur in an expression:

```
"price is $" + myCandy.getPrice()
```

- The result returned by the call to method **`getPrice()`** (type **`double`**) is appended to the string that starts with the characters **`price is $`**

Call to Constructor

```
public FoodItem(String desc, double aSize,
                double aPrice) {
   description = desc;
   size = aSize;
   price = aPrice;
}
```

- A constructor executes when a new object is created and initializes the object's data fields. A call to this constructor always follows the word **new** and has the form:

```
FoodItem myCandy = new FoodItem("Snickers",
                                6.5, 0.55);
```

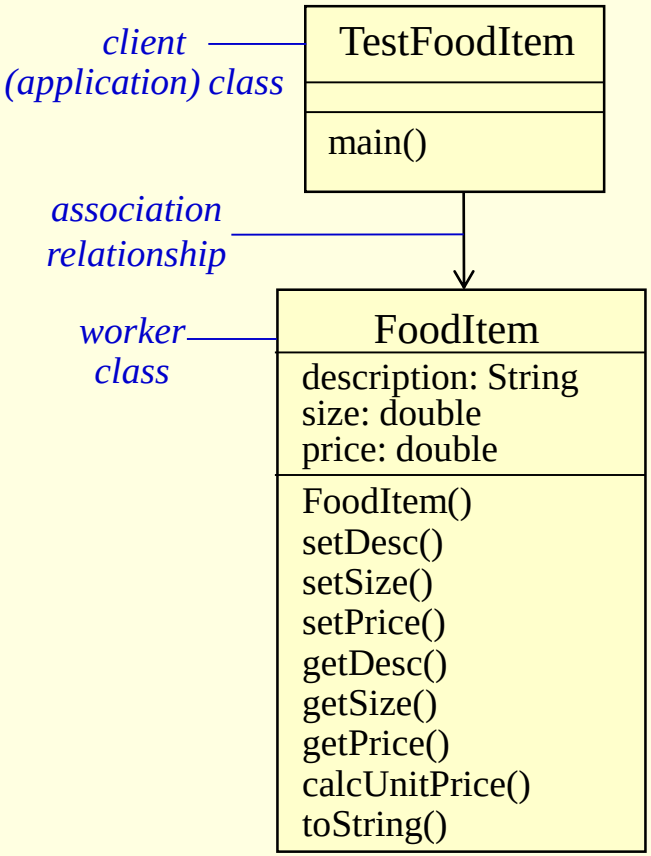
Argument/Parameter Correspondence for `FoodItem("Snickers", 6.5, 0.55)`

Argument	*Parameter*
`"Snickers"`	`desc (references "Snickers")`
`6.5`	`aSize (value is 6.5)`
`0.55`	`aPrice (value is 0.55)`

Primitive type arguments are passed by value which means the method cannot change the argument value.

Testing the class FoodItem

UML class diagram



Program classes and their relationships

Application or Client of FoodItem

```
/*
 * TestFoodItem.java
 * Tests class FoodItem
 */
public class TestFoodItem {

  public static void main(String[] args) {
    FoodItem myCandy = new FoodItem("Snickers", 6.5,
                                          0.55);
    FoodItem mySoup = new FoodItem(
                          "Progresso Minestrone",
                                 16.5, 2.35);
    System.out.println(myCandy.toString());
    System.out.println(" -- unit price is $" +
                       myCandy.calcUnitPrice());
    System.out.println(mySoup.toString());
    System.out.println(" -- unit price is $" +
                       mySoup.calcUnitPrice());
  }
}
```

Sample Run of Class TestFoodItem

TestFoodItem

```
Snickers, size : 6.5, price $0.55
 -- unit price is $0.08461538461538462
Progresso Minestrone, size : 16.5, price $2.35
 -- unit price is $0.14242424242424243
```

The `equals` Method

- The `Rectangle` class has an `equals` method.
- If we try the following:

```
Stock stock1 = new Stock("GMX", 55.3);
Stock stock2 = new Stock("GMX", 55.3);
if (stock1 == stock2) // This is a mistake.
  System.out.println("The objects are the same.");
else
  System.out.println("The objects are not the
  same.");
```

only the addresses of the objects are compared.